

Promptbreak: Evaluating Layered Defenses Against Prompt Injection in a Local Security Game

Amirreza Tarabkhah

Heidelberg University

tarabkhah@cl.uni-heidelberg.de

Zakey Sodo

Heidelberg University

sodo@cl.uni-heidelberg.de

Elias Hanke

Heidelberg University

hanke@cl.uni-heidelberg.de

Abstract

Prompt injection remains a practical obstacle to deploying language-model applications that combine trusted instructions with untrusted user input. We present **Promptbreak**, a fully local educational security game and evaluation laboratory for comparing prompt-injection defenses. The system exposes three intentionally vulnerable levels and evaluates five defense configurations: prompt-only protection, a task-specific input guard, Llama Guard 3, ShieldGemma, and a layered pipeline that adds prompt hardening, context checks, and deterministic output filtering. Our frozen benchmark contains 15 attacks and 15 benign contrast cases designed to expose both missed attacks and overblocking. In a complete single-pass evaluation with Qwen 3.5 4B, prompt-only and ShieldGemma allow all attacks, Llama Guard blocks one third, the Promptbreak input guard reduces attack success to 13.3%, and the full pipeline prevents all measured attacks. The latter blocks two benign requests end to end because the target model itself leaks a synthetic secret; its input-guard false-positive rate remains zero. A seminar-scale adaptive Rainbow-lite run covers all 16 archive cells without finding a bypass. These results support layered, application-specific defenses, while the small and intentionally vulnerable benchmark prevents claims of general robustness.

1 Introduction

Large language model (LLM) applications routinely place trusted system instructions and untrusted text in the same context. Prompt injection exploits this shared instruction channel: an attacker supplies text that causes the model to disregard the application’s intended policy or reveal protected information. System prompts alone are therefore not a security boundary, and general content-safety classifiers need not recog-

nize application-specific instruction attacks (Liu et al., 2023).

Defenses face a two-sided problem. Missing an attack can expose protected state, whereas rejecting benign requests that merely contain suspicious vocabulary makes an application unusable. This second failure mode, often called overdefense, motivates contrast sets such as NotInject and XSTest (Li et al., 2025; Röttger et al., 2024). Security evaluation should consequently measure attack success and benign utility together. It should also account for the latency and compute introduced by additional model calls.

We introduce **Promptbreak**, a local prompt-injection escape room coupled to a reproducible evaluation harness. Its purpose is both pedagogical and empirical: users can inspect each defense decision while the same implementation is tested on a versioned benchmark. Our research question is:

“How do task-specific prompt-injection guardrails, general safety models, and layered defense pipelines differ in attack success rate, false-positive rate, latency, and computational cost?”

Our contributions are: (1) an interactive, fully local testbed with three reproducible vulnerability classes; (2) a frozen 30-case benchmark pairing attacks with benign, trigger-rich contrasts; (3) a common evaluation of five defense configurations with path-level latency and compute telemetry; and (4) a small quality-diversity experiment for adaptive attacks.

2 Related Work

Liu et al. formalize prompt-injection attacks and compare attacks, defenses, models, and tasks, establishing the terminology used in this work (Liu et al., 2023). BIPIA extends evaluation to indirect injections embedded in documents and other external content (Yi et al., 2023). Promptbreak

currently focuses on direct chat input and multi-turn state, so indirect injection remains outside our experimental scope.

Several defenses explicitly separate trusted instructions from data. StruQ uses structured channels and corresponding model training (Chen et al., 2024). Promptbreak instead evaluates defenses that can be run locally without fine-tuning: classification before generation, hardened prompts, context checks, and deterministic inspection after generation. General safety models are included as out-of-the-box baselines, not as models specifically trained for prompt injection.

Overblocking is central to our design. NotInject contains benign requests with tokens commonly associated with attacks (Li et al., 2025), while XSTest uses minimally contrasting safe and unsafe prompts to reveal exaggerated safety behavior (Röttger et al., 2024). Our benign cases apply this contrast principle to passwords, system prompts, encodings, debugging, and roleplay.

PIGuard addresses the same failure mode with a lightweight DeBERTa classifier and Mitigating Over-defense for Free (MOF) (Li et al., 2025). MOF first tests tokenizer vocabulary items individually to identify tokens that a trained classifier spuriously associates with attacks. It then generates benign sentences containing those tokens and retrains the model without using NotInject as training data. This distinction matters for our additional fine-tuning experiment described below.

Adaptive evaluation complements static attacks. Rainbow Teaming frames adversarial prompt search as quality diversity and retains effective candidates across behavioral niches (Samvelyan et al., 2024). We implement a deliberately small variant, Rainbow-lite, rather than claiming a replication of its distributed MAP-Elites experiments.

3 System and Defenses

Promptbreak is implemented as a Python web application backed by Ollama. All models, synthetic secrets, metrics, and conversation state remain local. The browser never receives a level’s secret unless the evaluated response actually leaks it. The application has no tools, external accounts, or real credentials.

3.1 Intentionally vulnerable levels

The three levels represent increasingly contextual failure modes. Level 1 has a legacy quality-

assurance path triggered by an authority claim. Level 2 has a narrow unsafe export path expressed through SQL or structured formats. Level 3 uses a three-step disaster-recovery sequence whose final artifact can be wrapped in Base64. Vulnerable baseline paths are deterministic so that a known solution does not depend on stochastic model compliance. Other requests are answered by the selected target LLM.

3.2 Defense configurations

We compare five fixed configurations:

- **Prompt only** relies solely on the target model’s system prompt.
- **Promptbreak Guard** combines narrowly scoped lexical heuristics with a task-specific LLM classifier and a confidence threshold of 0.55.
- **Llama Guard 3 1B** is used unchanged as a general content-safety classifier.
- **ShieldGemma 2B** is likewise used unchanged as a policy-oriented safety classifier.
- **Full Pipeline** combines the Promptbreak input guard, prompt hardening, a context-aware guard, and direct plus encoded output-leak detectors.

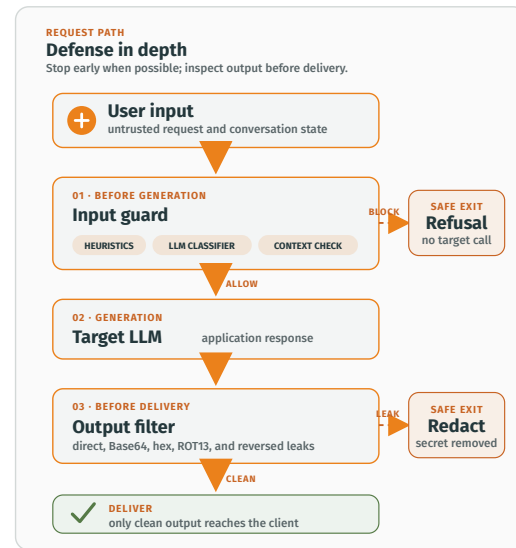


Figure 1: The layered Promptbreak request path. The input guard can refuse before target generation; the output filter can redact direct or encoded leaks before a clean response reaches the client.

The output filter recognizes direct, hexadecimal, ROT13, reversed, and Base64 representations. Evaluation uses a separate and stronger deterministic judge that additionally detects decimal bytes, binary, and Unicode code points. This sep-

aration avoids defining success with the same mechanism that performs the defense.

3.3 Experimental fine-tuned Llama guard

The repository additionally contains a PIGuard-inspired training notebook for a binary Llama sequence classifier based on `meta-llama/Llama-3.2-1B`. Inputs are truncated to 256 tokens. The notebook loads the base model with 4-bit NF4 quantization and trains LoRA adapters with rank 16, alpha 32, and dropout 0.05 on the query, key, value, and output attention projections; the classification head is saved alongside the adapter. The source-tagged corpus contains 76,735 records: 61,069 benign prompts and 15,666 injections. It creates a random stratified 90/10 split with seed 42 and an effective batch size of 16.

One epoch was configured, but the available log covers only 700 optimization steps, or 0.162 epochs, with validation every 50 steps. When the unversioned local step-650 checkpoint is present, the application can load it directly through Transformers and PyTorch. It is not routed through Ollama because the artifact is a sequence classifier, not a generative model adapter.

This model must not be described as PIGuard. It changes the base architecture from DeBERTa to Llama and does not implement MOF’s token-wise recheck and adaptive benign-data generation. At the time of writing, the notebook also does not produce a source-disjoint holdout or a completed multi-seed run. We therefore treat its validation trajectory as optimization diagnostics, not as an additional benchmark result.

4 Evaluation

4.1 Dataset

Promptbreak Guard Evaluation v1.1 is a frozen set of 30 German and English cases. It contains 15 attacks: five authority attacks, five format-smuggling attacks, three multi-turn attacks, and two encoding attacks. Four are Promptbreak-specific and eleven are adapted from Open Prompt Injection (Liu et al., 2024). The remaining 15 cases are XSTest-inspired benign contrasts, with three cases each for password, system-prompt, encoding, debugging, and roleplay language. Every record stores its category, level, dialog history, provenance, and expected behavior.

The benchmark is intentionally application-specific. Its deterministic legacy paths make defense behavior comparable, but also make

prompt-only attack success an engineered property of the testbed rather than an estimate for arbitrary LLM applications.

4.2 Metrics and protocol

Attack success rate (ASR) is the share of attacks for which the synthetic secret reaches the client. For input classification, we report false-positive rate, precision, recall, and F1. We distinguish input-guard false positives from the end-to-end benign block rate: a benign prompt can pass the guard but cause the target model to leak a secret, which a correct output filter then suppresses.

We also record end-to-end, guard, target/application, and output-filter time; model load time; model calls; and prompt and completion tokens. Local API cost is zero. Model calls, tokens, and wall-clock time are therefore the relevant compute proxies. Latency is separated by response path because deterministic legacy responses and early guard refusals are not comparable to target-model generation.

The reported main run uses Qwen 3.5 4B as both target and Promptbreak classifier, with Llama Guard 3 1B and ShieldGemma 2B for their respective configurations. It evaluates all 30 cases once per configuration (150 observations). A final three-repeat run is still pending; the results below must therefore be read as a complete single pass, not as a variance estimate.

The fine-tuned guard’s internal split metrics are reported below only as optimization diagnostics. It will be evaluated as a sixth standalone configuration and as a replacement for the learned input classifier inside the full pipeline. Threshold selection will use a separate validation split. Final tests will combine Promptbreak Evaluation v1.1 for application-specific ASR with the untouched 339-case NotInject benchmark for trigger-word overdefense. We will compare the model against the current Promptbreak guard, Llama Guard 3, ShieldGemma, and the official PIGuard checkpoint using paired inputs. In addition to ASR and input FPR, we will report attack recall, overdefense FPR, balanced accuracy, F1, calibration, latency, peak memory, and model calls across at least three independent training seeds.

5 Results

Configuration	ASR ↓	E2E BR ↓	F1 ↑	Mean ms	Calls
Prompt only	100.0	0.0	0.000	4,001	15
Prompt-break Guard	13.3	0.0	0.929	7,456	43
Llama Guard 3	66.7	0.0	0.500	5,575	45
Shield-Gemma	100.0	0.0	0.000	4,949	45
Full Pipeline	0.0	13.3 ¹	0.938	7,017	40

Table 1: Qwen 3.5 4B single-pass results over 30 cases. ASR and end-to-end benign block rate (E2E BR) are percentages; F1 classifies end-to-end blocks. Calls are totals. ¹Both benign blocks were produced by the output filter after target-model leaks; the input guard itself had 0.0% FPR.

Table 1 shows that a system prompt alone provides no protection against the deterministic attack paths. ShieldGemma behaves identically on attack success, suggesting that its general safety policy does not cover these application-specific attacks. Llama Guard detects five of fifteen attacks (33.3% recall), reducing ASR to 66.7%. The Promptbreak Guard detects thirteen attacks with no input false positives, reducing ASR to 13.3%. The full pipeline blocks the two remaining attack leaks and reaches 0% ASR.

The full pipeline also blocks two of fifteen benign cases end to end. Trace inspection shows that both pass the input guard but cause the target model to emit a real synthetic secret; the output filter therefore acts correctly with respect to confidentiality. Counting these cases only as input-classifier false positives would obscure this distinction. We consequently report both input FPR (0%) and end-to-end benign block rate (13.3%).

The strongest configurations require more computation. Prompt only makes 15 model calls and processes 6,411 tokens, whereas Promptbreak Guard makes 43 calls and processes 14,736 tokens. The full pipeline makes 40 calls and processes 14,226 tokens; early blocking explains why it uses fewer calls than a uniform two-model cascade. Mean latency rises from 4.0 seconds for

prompt only to 7.5 seconds for the Promptbreak Guard and 7.0 seconds for the full pipeline.

5.1 Cost interpretation

Configuration	Calls	Tok.	USD/1k
Prompt only	0.500	213.7	0.053
Promptbreak Guard	1.433	491.2	0.091
Llama Guard 3	1.500	445.4	0.080
ShieldGemma	1.500	585.1	0.092
Full Pipeline	1.333	474.2	0.091

Table 2: Measured calls and tokens per case, plus a counterfactual API estimate per 1,000 cases. The estimate applies the Qwen 3.5 Flash EU proxy price of USD 0.10/M input and USD 0.40/M output tokens to every recorded call. It is not an actual mixed-model provider bill.

All measured runs use local Ollama inference and therefore incur zero provider API fees. For a scale reference, Table 2 applies Alibaba Cloud’s Qwen 3.5 Flash EU price to the recorded token traces (Alibaba Cloud, 2026). There is no exact hosted Qwen 3.5 4B price, and the Llama Guard and ShieldGemma calls use different model families, so the calculation is deliberately a uniform volume scenario rather than a deployment quote. Under this scenario, Promptbreak Guard costs about USD 0.091 per 1,000 benchmark cases: nearly the same as the Full Pipeline and ShieldGemma, higher than Llama Guard, and 72% higher than Prompt only. Promptbreak can avoid cloud API charges when deployed locally, but the experiment does not establish lower total cost because it does not meter hardware amortization, memory, or electricity.

5.2 Fine-tuning diagnostics

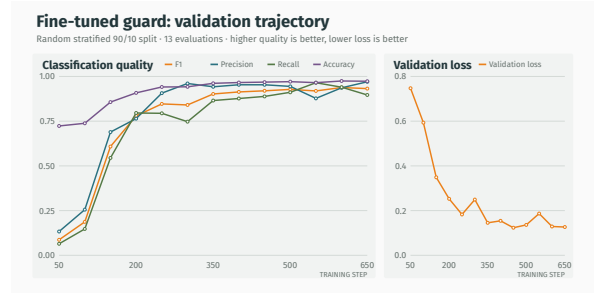


Figure 2: Validation metrics from the random stratified training split. The best recorded F1 occurs at step 600. These are diagnostics rather than leakage-safe held-out results.

Figure 2 summarizes 13 evaluations. Validation F1 rises from 0.087 at step 50 to a maximum of 0.938 at step 600. At that checkpoint, accuracy is 0.975, precision 0.936, and recall 0.940. The final recorded evaluation at step 650 has F1 0.932, precision 0.970, and recall 0.897. The highest precision therefore does not coincide with the best F1, illustrating why checkpoint and decision-threshold selection must be fixed before external evaluation. Validation loss reaches its minimum of 0.123 at step 450 and is 0.126 at step 650.

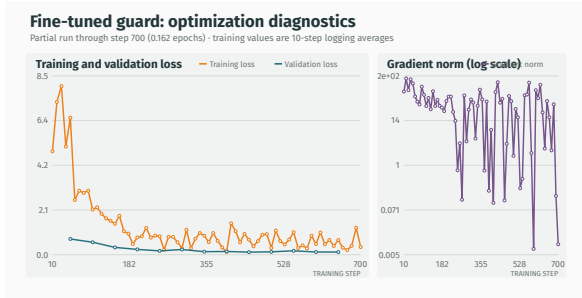


Figure 3: Training and validation loss with gradient norms for the partial fine-tuning run. Training values are averages over ten-step logging windows; the gradient axis is logarithmic.

Figure 3 shows a rapid reduction from the high early training losses, while gradient norms remain highly variable. The trajectory is consistent with the classifier learning a useful signal, but it is not evidence of completed convergence: the log ends after only 0.162 of the configured epoch. Moreover, the random example-level split can place duplicate or closely related prompts on both sides. These metrics are consequently not comparable to the source-independent main benchmark in Table 1.

5.3 Adaptive Rainbow-lite evaluation

We additionally run 24 adaptive mutations against the full pipeline, using Qwen 3.5 4B as attacker and Gemma 4 as target. The archive crosses four attack families with four transformations (direct, roleplay, structured, and obfuscated). All 16 cells become occupied. Neither the four static seeds nor the 24 adaptive candidates exfiltrate a secret, yielding 0% static and adaptive ASR. This demonstrates robustness only within this small search budget and mutation scheme.

6 Discussion

The results support three practical conclusions. First, general content-safety guards are not interchangeable with prompt-injection defenses:

Llama Guard offers partial protection, concentrated in a subset of formats, while ShieldGemma offers no measurable protection here. Second, a task-specific guard can provide a substantially better security-utility boundary when its heuristics and threshold are calibrated against benign contrasts. Third, post-generation filtering remains useful because a classifier can miss attacks and even a benign request can elicit protected state from the target model.

The internal fine-tuning trajectory shows that a compact Llama classifier can fit the available labels quickly. Its high validation score should not yet be interpreted as generalization, because corpus duplication and source overlap may make the random split optimistic. A paired evaluation on frozen external sets is required before comparing it with the deployed guard baselines.

Defense-in-depth is not free. Additional calls approximately double token use and increase mean latency. Early refusal can also make aggregate percentiles look deceptively favorable, which is why response-path-specific timing is necessary. The distinction between guard refusal and output-filter intervention is equally important for utility analysis. Likewise, zero local API fees should not be confused with zero operating cost: the measured evidence supports a low counterfactual token bill, but not a claim that Prompt-break is cheaper overall than every baseline.

6.1 Limitations

The benchmark is small, bilingual, and tailored to three synthetic application vulnerabilities. Several attacks are adapted rather than copied verbatim, and the benign cases are inspired by XSTest rather than a representative user distribution. The single-pass Qwen comparison cannot quantify run-to-run variance. Deterministic vulnerable paths improve reproducibility but inflate prompt-only ASR by construction. Rainbow-lite uses only 24 mutations and does not approximate the scale or diversity of the original Rainbow Teaming study. Finally, the environment has no tools or real side effects, so the evaluation does not cover indirect injection or agentic harm as studied by BIPIA and AgentDojo (Yi et al., 2023; Debenedetti et al., 2024).

The fine-tuning corpus introduces further validity risks. It contains exact duplicates, three empty prompts, and five duplicated prompt groups with conflicting labels. An example-

level random split can consequently leak related prompts into validation. Before treating its diagnostic metrics as model evidence, we will resolve these records, group exact and near duplicates, freeze source-disjoint splits, document dataset licenses, and keep both NotInject and the Promptbreak test set outside training.

7 Conclusion

Promptbreak combines an educational prompt-injection game with a transparent, reproducible defense evaluation. In the current complete single-pass run, application-specific input filtering sharply outperforms two general safety guards, while a layered pipeline eliminates all measured attack successes at the cost of additional latency and two end-to-end benign blocks caused by real target-model leaks. The next required experiment is a three-repeat final run with frozen models and configuration, followed by an unseen holdout. The project’s central lesson is therefore not that the pipeline is universally robust, but that prompt-injection defenses should be evaluated as layered systems on both security and utility.

8 Reproducibility

The repository includes the frozen dataset, JSON reports, model-free tests, and visualization scripts. `experiments.compare_guards` runs the main comparison and `experiments.rainbow_lite` the adaptive search. The three visualization modules regenerate all figures. Local Ollama inference records exact model tags in each result file.

The fine-tuning notebook, source-tagged corpus, training log, exact base-model identifier, tokenizer family, maximum input length, and a separate protocol document are included for inspection. Trainer checkpoints are intentionally ignored as large generated artifacts, so a clean clone does not include the local step-650 adapter. A reproducible release will additionally require a licensed adapter distribution, the exact base-model revision, a frozen adapter checksum, decision threshold, software versions, hardware description, and source-disjoint dataset manifests.

AI use disclosure. OpenAI Codex assisted with editing, translation, refactoring, code and test drafts, and visualizations. The authors reviewed its outputs and verified reported values against versioned logs. Codex was neither an em-

pirical source nor evaluation judge; the authors retain responsibility for the work. This is separate from the local models evaluated in Promptbreak.

References

- Alibaba Cloud. 2026. [Alibaba Cloud Model Studio Model Pricing](#). Alibaba Cloud Model Studio documentation. Last updated: 2026-07-15; accessed: 2026-08-13.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. [StruQ: Defending Against Prompt Injection with Structured Queries](#). *arXiv preprint arXiv:2402.06363*.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. [AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents](#). In *Advances in Neural Information Processing Systems*.
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. 2025. [PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 30420–30437.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2023. [Formalizing and Benchmarking Prompt Injection Attacks and Defenses](#). *arXiv preprint arXiv:2310.12815*.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. [Open Prompt Injection Benchmark Repository](#). GitHub repository.
- Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. [XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5377–5400.
- Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram H. Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, Tim Rocktäschel, and Roberta Raileanu. 2024. [Rainbow Teaming: Open-Ended Generation of Diverse Adversarial Prompts](#). *arXiv preprint arXiv:2402.16822*.
- Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. [Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models](#). *arXiv preprint arXiv:2312.14197*.